Academic Research Paper Repository with Semantic Search

DBMS Experiential Learning – Level 3 | Revised Project Proposal
Sonia Sharma (PES1UG24AM904) | P Saanvi (PES1UG24AM905)
29 October 2026

1. Introduction

A web platform to store, search and discover academic research papers using both keyword search and meaning-based (semantic) search, so that relevant papers can be found even when the exact keywords do not match. The platform also models authors, citations and conferences as first-class relationships rather than isolated records.

2. Problem Statement

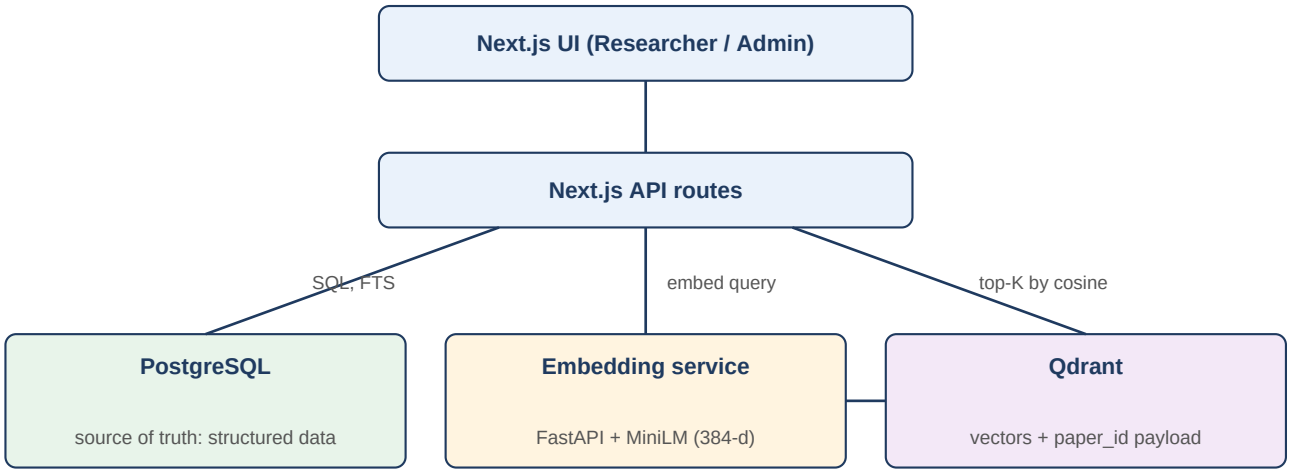
Keyword-based search fails when a user phrases a query differently from a paper's actual wording, even when the topic is the same (for example, "teaching machines to understand human language" will not match a paper that only says "natural language processing"). Most repositories also treat papers as flat records and do not capture author, citation and conference relationships in a properly constrained relational model.

3. Objectives

- Design a normalised relational database (up to 3NF) for papers, authors, citations, conferences, users and bookmarks, with full integrity constraints.
- Support keyword/full-text search, semantic (vector-based) search over abstracts, and a hybrid mode that fuses both.
- Provide citation information, trigger-maintained citation counts and a statistics dashboard.
- Run on a real dataset of 2,000 to 5,000 papers, not hand-typed samples.
- Justify the combined use of a relational database and a separate vector database, and show how consistency between them is maintained.
- Demonstrate advanced DBMS concepts beyond CRUD: transactions, triggers, views, indexing, full-text search and query-plan analysis.

4. Proposed Solution and Architecture

PostgreSQL stores all structured data and is the single source of truth. **Qdrant** (a dedicated vector database) stores embeddings of paper abstracts, with the paper ID as payload. A small Python **embedding service** (sentence-transformers, model all-MiniLM-L6-v2, 384 dimensions) turns the user's natural-language query into a vector. Qdrant returns the closest paper IDs, and the application then fetches the full records from PostgreSQL. A **Next.js** app with API routes connects everything.



Search modes

- **Keyword:** PostgreSQL full-text search using a weighted tsvector (title weight A, abstract weight B), websearch_to_tsquery and ts_rank_cd ranking.
- **Semantic:** query embedding, then Qdrant top-K by cosine similarity, then hydrate the results from PostgreSQL.
- **Hybrid:** run both and merge with Reciprocal Rank Fusion, score = sum of 1 / (60 + rank) across the two ranked lists.

5. Key Features

- Role-based login (Researcher, Admin) with hashed passwords.
- Add and manage papers, authors and conferences (admin); atomic paper insertion.
- Keyword, semantic and hybrid search with pagination and year filter.
- Paper detail page with authors, conference, cited-by and references lists.
- Citation tracking with trigger-maintained counts.
- Bookmarks for researchers.
- Statistics dashboard: top cited papers, papers per year, conference and author statistics.

6. Technology Stack

Layer	Choice	Note
Frontend	Next.js	Search, detail, dashboard and admin screens
Backend	Next.js API routes	Thin layer over both databases
SQL database	PostgreSQL 16	Source of truth, full-text search, triggers, views
Vector database	Qdrant (Docker)	Chosen over pgvector so the two-database design is real. pgvector is documented in the report as the considered alternative.
Embeddings	Python, sentence-transformers, FastAPI	Bulk ingestion script plus a small live query-embedding endpoint
Dataset	OpenAlex API	Free, no key; abstracts rebuilt from the inverted index
Environment	Docker Compose	Identical Postgres and Qdrant setup for both teammates
Version control	Git and GitHub	Numbered SQL migrations, one branch per task

7. Database Design

7.1 Entities and constraints

Table	Key attributes	Constraints
users	user_id, name, email, password_hash, role, created_at	PK; email UNIQUE; role CHECK in (researcher, admin)
conferences	conference_id, name, acronym, year, location	PK; UNIQUE(name, year)
papers	paper_id, title, abstract, year, doi, conference_id, citation_count, embedding_status, search_tsv, created_at	PK; title NOT NULL; doi UNIQUE; citation_count DEFAULT 0; embedding_status CHECK in (pending, done, failed); conference_id FK ON DELETE SET NULL
authors	author_id, name, affiliation	PK; name NOT NULL
paper_authors	paper_id, author_id, author_order	Composite PK; both FKs ON DELETE CASCADE (resolves the M:N relationship)
citations	citing_paper_id, cited_paper_id	Composite PK; both FKs ON DELETE CASCADE; CHECK (citing_paper_id <> cited_paper_id)
bookmarks	user_id, paper_id, created_at	Composite PK; both FKs ON DELETE CASCADE
audit_log (optional)	log_id, action, paper_id, admin_id, logged_at	Records admin deletions, filled by a trigger

7.2 DBMS concepts and where they are used

Concept	Where it is used	Why
Transactions	add_paper(): insert paper, link or create authors, add citations	All-or-nothing insert; a deliberate failure is shown in the demo
Triggers	AFTER INSERT / DELETE on citations updates papers.citation_count; audit trigger on paper deletion	A maintained counter avoids COUNT(*) on every read
Views	v_paper_details, v_top_cited_papers, v_papers_per_year, v_author_stats, v_conference_stats	Feed the dashboard and detail page directly
Indexing	GIN on search_tsv; B-tree on papers(year), papers(conference_id), citations(cited_paper_id), paper_authors(author_id); optional pg_trgm on authors.name	Proved with EXPLAIN ANALYZE before and after
Full-text search	Generated weighted tsvector column	Titles rank above abstracts
Vector similarity	Qdrant collection, 384-d, cosine	Finds conceptually related papers
Normalisation	M:N tables for authors and citations	3NF, no repeating groups

7.3 Why two databases

SQL handles structured, relationship-heavy data that needs joins, constraints, transactions and triggers. The vector database handles unstructured text meaning, using purpose-built approximate nearest neighbour search (HNSW) and payload filtering. Each does what it is best at, and neither can replace the other cleanly. Keeping the vectors in a separate Qdrant instance, rather than a pgvector column, makes this separation explicit and gives a genuine integration problem to solve and defend.

7.4 Keeping PostgreSQL and Qdrant consistent

PostgreSQL is the source of truth. Qdrant is written only after the SQL transaction commits:

1. Insert the paper inside a transaction with embedding_status = 'pending'.
2. After commit, embed title + abstract and upsert to Qdrant with id = paper_id.
3. On success set embedding_status = 'done'; on failure set 'failed'.
4. A retry script reprocesses every 'pending' or 'failed' row.
5. On deletion, remove the row from PostgreSQL first, then delete the Qdrant point. Any orphan vector is harmless because search results are always re-fetched from PostgreSQL and missing IDs are dropped.

8. Dataset

2,000 to 5,000 papers from a focused area (for example NLP and machine learning, so semantic queries are impressive), pulled from the OpenAlex API with authors, venues and referenced works. Abstracts are rebuilt from OpenAlex's inverted-index format. Papers without abstracts are skipped, and citation edges are kept only when both papers exist in the dataset so that foreign keys hold. The ingestion script is re-runnable (ON CONFLICT DO NOTHING).

9. API Contract (frozen at the start of Part 1)

Field names are agreed up front. Part 1 implements the read endpoints and the search endpoint; Part 2 builds the UI and the write endpoints against working APIs. Changes after handoff are agreed by both teammates first.

```
GET  /api/search?q=...&mode=keyword|semantic|hybrid&page=1      [Part 1]
     -> { results: [{paper_id, title, abstract_snippet, year, conference,
                     authors[], citation_count, score}], total }
GET  /api/papers/:id        -> full paper + authors + conference + cited/citing   [Part 1]
POST /api/papers            -> atomic add via add_paper() (admin only)            [Part 2]
GET  /api/stats/overview    -> top cited, papers per year, conference stats       [Part 1]
```

10. Work Division and Handoff Plan

The work is split 50/50 by effort and runs as a handoff. **Part 1 (P Saanvi) is the relational database half** and is completed first. **Part 2 (Sonia Sharma) is the vector database half** and starts once Part 1 passes the handoff gate below. Each part totals 8.0 working days. Because the vector work searches the papers that Part 1 loads, the order is sequential by design.

	Owner	Scope	Effort	Share
Part 1	P Saanvi	RDBMS: infrastructure, schema, indexes, triggers, views, transactions, OpenAlex ingestion, keyword search, authentication, CRUD / bookmark / statistics APIs, query-plan evidence	8.0 days	50%
Part 2	Sonia Sharma	Vector DB: embedding service, Qdrant collection and pipeline, PostgreSQL-Qdrant consistency, semantic and hybrid search, Next.js front end, semantic-vs-keyword evidence, final testing and demo	8.0 days	50%

The Next.js front end sits in Part 2 because Part 1 already carries the heavier backend work (ingestion, authentication and all non-vector APIs). Part 1 exposes everything the front end needs as documented API routes.

ID	Task	Owner	Days	After	Deliverable
PART 1 - RDBMS (P Saanvi), 1A: Foundation (Days 1-2)					
S1	Create the GitHub repo (/db, /ingest, /web, /docs). Write docker-compose.yml with Postgres 16 and Qdrant so both services start for Part 2. Write the .env.example and publish the API contract	Saanvi	0.5	Start	docker compose up works
S2	Schema DDL with all constraints as numbered migrations (001_schema.sql and so on); ER diagram	Saanvi	1.0	S1	Database rebuilds from scratch
PART 1 - RDBMS, 1B: Database features (Days 3-4)					
S3	Indexes: GIN on search_tsv, B-tree on FK and year columns, optional pg_trgm; note why each exists	Saanvi	0.5	S2	002_indexes.sql
S4	Triggers: citation_count increment and decrement, audit log on paper deletion; test script	Saanvi	0.5	S2	Counter test passes
S5	Views: v_paper_details, v_top_cited_papers, v_papers_per_year, v_author_stats, v_conference_stats	Saanvi	0.5	S2	Five working views
S6	Transactional add_paper() with rollback demo; sets embedding_status = pending	Saanvi	0.5	S2	Atomic insert verified
PART 1 - RDBMS, 1C: Data and APIs (Days 5-8)					
S7	OpenAlex ingestion script: papers, authors, venues, citations; reconstruct abstracts; batch inserts; re-runnable. All rows start as embedding_status = pending	Saanvi	1.5	S3-S6	2,000+ papers loaded
S8	Keyword search function (websearch_to_tsquery, ts_rank_cd) and /api/search?mode=keyword	Saanvi	0.5	S7	Keyword search works
S9	Authentication API: register, login, password hashing, sessions, role-based guard (Researcher / Admin)	Saanvi	1.0	S2	Protected routes work
S10	API routes for papers CRUD (create calls add_paper), authors, conferences, bookmarks, /api/papers/:id and /api/stats/overview	Saanvi	1.0	S5, S6, S9	All non-vector APIs documented
S11	EXPLAIN ANALYZE before and after the GIN index, RDBMS sections of the report, README and handoff note	Saanvi	0.5	S8, S10	Evidence in /docs, handoff ready
HANDOFF GATE: Part 1 is signed off against the checklist below					
PART 2 - Vector DB (Sonia), 2A: Vector pipeline (Days 9-11)					

ID	Task	Owner	Days	After	Deliverable
V1	Embedding service: FastAPI with sentence-transformers (all-MiniLM-L6-v2, 384-d), POST /embed	Sonia	0.5	Handoff	Service returns 384-d vectors
V2	Create the Qdrant collection (384-d, cosine, id = paper_id); bulk-embed title + abstract in batches; set embedding_status = done	Sonia	1.0	V1	All papers embedded
V3	Consistency: post-commit upsert after add_paper, delete sync, failed-status handling, retry script, orphan-safe hydration	Sonia	1.0	V2	Both databases stay in sync
PART 2 - Vector DB, 2B: Semantic and hybrid search (Days 11-12)					
V4	Semantic search: embed query, Qdrant top-K, hydrate from PostgreSQL; /api/search?mode=semantic	Sonia	1.0	V2	Semantic search works
V5	Hybrid search: Reciprocal Rank Fusion over keyword (S8) and semantic (V4) results; /api/search?mode=hybrid	Sonia	0.5	V4	Three modes available
PART 2 - Vector DB, 2C: Front end (Days 13-15)					
V6	Next.js scaffold, layout, search page with mode toggle, year filter, pagination and result cards	Sonia	1.5	V5	Search UI on real data
V7	Login and role-based screens, paper detail page, bookmarks, admin CRUD screens (all over the APIs from S9 and S10)	Sonia	1.0	V6	End-to-end user flows
V8	Statistics dashboard with charts from /api/stats/overview	Sonia	0.5	V6	Dashboard live
PART 2 - Vector DB, 2D: Finish (Day 16)					
V9	5-6 semantic-versus-keyword examples, vector and consistency sections of the report, end-to-end testing, demo script	Sonia	1.0	V7, V8	Final demo-ready build

Blue rows are Part 1 (Saanvi, 8.0 days); green rows are Part 2 (Sonia, 8.0 days). Day numbers are working days from the start and are approximate.

10.1 Handoff gate: what Part 1 delivers

Item	What Saanvi hands over	How Sonia checks it
Environment	docker-compose.yml starting Postgres and an empty Qdrant, .env.example	docker compose up, then connect to both
Schema	Numbered migrations, ER diagram	Rebuild the database from scratch
Data	2,000+ papers with authors and citations, all embedding_status = pending	Count rows; count by embedding_status
Hooks for the vector side	embedding_status column, add_paper() setting pending, audit trigger on paper deletion, search_tsv for keyword ranking	Insert a test paper and confirm status = pending
APIs	Keyword search, paper detail, stats, authentication, CRUD and bookmark routes, documented	Call each route with curl or a saved request collection
Documents	README with run steps, API contract, EXPLAIN ANALYZE evidence, test scripts	Follow the README on a clean checkout

10.2 Interface agreed up front for the vector side

- Qdrant collection named **papers**: 384 dimensions, cosine distance, point id = paper_id, payload {paper_id, year}.
- Embedding input text is title + ". " + abstract; the service exposes POST /embed returning a 384-d vector.
- Semantic search returns an ordered list of paper_ids with scores; full records are always re-fetched from PostgreSQL with the same query the keyword search uses.

- Hybrid search fuses the keyword ranking from S8 with the semantic ranking from V4 using Reciprocal Rank Fusion.

Working rules

- One Git branch per task ID, merged to main only when the deliverable works.
- SQL changes go in as new numbered migration files, never edits to applied ones. After handoff, Sonia raises any needed schema change with Saanvi first.
- The handoff is a short walkthrough (20 minutes): run the checklist together, then Part 2 starts.
- If Sonia is blocked by something in Part 1, Saanvi fixes it the same day.
- If the deadline is tight, Sonia can start V1 and V6 early, in parallel with Part 1: they need only the Docker setup (S1) and the API contract, and this saves about two days.

11. Risks and Mitigations

Risk	Mitigation
PostgreSQL and Qdrant drift out of sync	embedding_status column, post-commit upsert, retry script, results always hydrated from PostgreSQL
OpenAlex abstracts arrive as inverted indexes	Reconstruction function, skip papers without abstracts
Part 2 blocked by missing setup details after handoff	Handoff gate checklist, README and handoff note with curl examples, database dump, 20-minute walkthrough call
Citation foreign-key failures during load	Keep only edges whose both endpoints are in the loaded set
Handoff delays the vector half	Handoff gate checklist; Sonia starts V1 and V6 early if time is short
Embedding model download or speed issues	Small 384-d model, batch encoding, cache the model in the Docker image

12. Expected Outcome

A working web application that lets users search and discover research papers by keyword, natural-language and hybrid queries over a real dataset of at least 2,000 papers. The project demonstrates a justified SQL plus vector database integration with a defined consistency strategy, and advanced DBMS features that go beyond basic CRUD: constraints, triggers, views, transactions, indexing with measured query plans, full-text search and vector similarity search.

Final deliverables checklist

- Repository with migrations, ingestion scripts and the Next.js app.
- ER diagram and relational schema with normalisation notes.
- EXPLAIN ANALYZE screenshots and semantic-versus-keyword examples.
- Trigger, view and transaction test evidence (including a deliberate rollback).
- Project report and a rehearsed demo script.